Operating Systems

ASSIGNMENT 1

CO1: *Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.*

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/joshna07790/CSA0404OPERATINGSYSTEM.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Operating Systems for a Smart Hospital

A 500-bed hospital is replacing its computer systems. It needs: bedside patient monitors that must raise an alarm within 10 milliseconds of an abnormal reading; nurses' handheld tablets; doctors' desktop workstations; a central database server for patient records that must never stop; an imaging server that processes MRI scans using many processors; and a link to two partner hospitals that share records.

The IT team has proposed using one general-purpose operating system everywhere. The clinical staff are concerned that alarms may be delayed, the database may stop during updates, and the tablets' batteries will not last a shift.

The number of devices may grow:

1 ward (40 monitors, 10 tablets) → 1 hospital (500 monitors, 120 tablets, 80 workstations) → a group of 3 hospitals sharing records

You are required to evaluate the requirements of every part of the system, judge which type and structure of operating system suits each, examine how programs obtain OS services through system calls, and defend a complete operating-system plan for the hospital.

Design Challenge:

The hospital is adding remote patient monitoring at home and a cloud-based records service.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate the Requirements of Each Computing Environment — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate what each part of the hospital needs from its operating system.

Discuss:

- Response-time, reliability, throughput, battery and security requirements of each device
- Which requirements conflict (for example: fast response and many users on one machine)
- Which requirements are critical to patient safety
- How the lessons from the history of operating systems (batch, time-sharing, personal, mobile) apply to each environment

You are expected to **rank the requirements of each environment and justify the ranking**.

Alternative Concept and Complexity Analysis

If the whole hospital used a single general-purpose operating system on every device, instead of a different type of OS for each environment, what would the solution be? Evaluate the worst-case alarm response time (unbounded scheduling delay versus a guaranteed deadline), the memory needed on each device, and the administration effort.

Evidence Expected

A ranked requirement table for every environment with a justification for each ranking.

TASK 2 – Judge the Type of Operating System for Each Environment 3 Marks

Bloom's Level: K5 – Evaluate

Compare the types of operating systems for the hospital.

Discuss:

- Mainframe, desktop, multiprocessor, distributed, clustered, real-time (hard and soft) and handheld systems
- Which type best fits the monitors, tablets, workstations, database server, imaging server and partner link
- Symmetric versus asymmetric multiprocessing for the imaging server • A clustered system with fail-over for the database server

Alternative Concept and Complexity Analysis

If the database server used a single large multiprocessor machine instead of a cluster of smaller machines with fail-over, what would the solution be? Evaluate the availability (a single point of failure versus fail-over within seconds), the cost, and how throughput grows with the number of processors (time) and the memory available (space).

Evidence Expected

A justified choice of operating-system type for each environment, with the rejected alternatives and reasons.

TASK 3 – Evaluate Operating-System Structures 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate the possible structures of the operating system.

Discuss:

- Simple (monolithic), layered, microkernel, modular (loadable kernel modules) and hybrid structures
- The effect of each structure on reliability, performance and ease of updating
- Which structure suits the patient monitors and which suits the database server
- How Linux, Windows, Android and a real-time OS such as FreeRTOS or QNX are structured

Alternative Concept and Complexity Analysis

If the bedside monitors used a monolithic kernel instead of a microkernel, what would the solution be? Evaluate the time cost of a service request (one system call versus message passing between servers), the memory taken by the kernel, and the effect of a faulty driver on the whole system.

Evidence Expected

A comparison of structures against the hospital's requirements, with a defended choice for two environments.

TASK 4 – Evaluate the Services an Operating System Must Provide 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate the services each environment needs.

Discuss:

- User interface, program execution, I/O operations, file systems, communication, error detection, resource allocation, accounting, protection and security
- Which services can be removed from the bedside monitor to make it smaller and faster
- Which services are essential for the database server
- How the OS protects one patient's records from another user's programs

Alternative Concept and Complexity Analysis

If the tablets used a full desktop operating system with every service enabled instead of a reduced handheld OS, what would the solution be? Evaluate the memory and storage used by the extra services (space), the start-up time and the battery life (time).

Evidence Expected

A service table for each environment with justified inclusions and exclusions.

TASK 5 – Examine System Calls 4 Marks

Bloom's Level: K5 – Evaluate

Examine how a program obtains OS services.

Discuss:

- The categories of system calls (process control, file management, device management, information maintenance, communication, protection)
- What happens step by step when a program calls read(): library call, trap, mode switch, kernel work, return
- How parameters are passed to the kernel (registers, a block in memory, the stack) • How strace shows the system calls made by a program

Alternative Concept and Complexity Analysis

If programs called the C library functions fopen, fread and fwrite instead of the open, read and write system calls directly, what would the solution be? Evaluate the number of system calls for a file of n bytes (about n / 4096 with library buffering versus n / b for a buffer of b bytes) and the extra buffer space used by the library.

Evidence Expected

A step-by-step explanation of a system call with strace evidence from a real program.

TASK 6 – Evaluate the Cost of System Calls 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the number of system calls affects performance.

Discuss:

- Why each system call costs much more than an ordinary function call
- How the buffer size changes the number of read() and write() calls when the database copies a file
- How library buffering (fread, fwrite) reduces the number of system calls
- The best buffer size for the hospital's backup program, using your measurements

Complete the Code

Complete TODO 1 below using the read() and write() system calls, and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
bufsize bytes. Returns the number of read() calls made, or -1 on error. */ long
copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];     int in = open(src, O_RDONLY);     if
(in < 0) return -1;     int out = open(dst, O_WRONLY | O_CREAT
| O_TRUNC, 0644);     if (out < 0) { close(in); return -1; }
long calls = 0;     ssize_t n;
    // TODO 1: Loop: read up to bufsize bytes into buf (count every read call in calls);
//   stop when read returns 0 (end of file) or -1; write exactly the n bytes that     //
were read

    close(in);
close(out);
    return calls; }

int main(void) {
    FILE *f = fopen("records.dat", "w");     /* a 10,000-byte test file */
for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);     fclose(f);

    size_t sizes[] = {1, 512, 4096};     for (int i = 0; i < 3; i++)
printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
copy_file("records.dat", "copy.dat", sizes[i]));
unlink("records.dat");     unlink("copy.dat");     return 0;
}
```

Expected output:

```
buffer    1 bytes: 10001 read() calls buffer
512 bytes:    21 read() calls buffer 4096
bytes:     4 read() calls
```

Run the program under strace -c and compare the number of read() calls reported with the program's output.

Alternative Concept and Complexity Analysis

If the backup program used memory-mapped files (mmap) instead of read() and write() with a buffer, what would the solution be? Evaluate the number of system calls (a few versus about n / b), the copying between kernel and user space, and the memory used for very large files.

Evidence Expected

Measured system-call counts and times for different buffer sizes, with a justified recommendation.

TASK 7 – Evaluate Process Control System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how a command shell on the database server starts and manages programs.

Discuss:

- What fork(), exec(), wait() and exit() each do
- How a shell runs a command, and how it obtains the command's exit status
- What a zombie and an orphan process are, and how the OS deals with them
- Why fork() followed by exec() is used instead of a single "create process" call

Complete the Code

Complete TODO 1 below using fork(), execlp() and waitpid(), and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>


int main(void)
{
    printf("shell: running job 1\n");
    fflush(stdout);                              /* empty the buffer before fork() */
// TODO 1: Create a child with fork(). In the child, replace the program with
    //   execlp("expr", "expr", "6", "*", "7", (char *)NULL) and exit(127) if exec
    //   fails. In the parent, wait for the child with waitpid() and print "job 1 exit
//   status N" using WEXITSTATUS



    printf("shell: running job 2\n");     fflush(stdout);     pid = fork();     if
(pid == 0) _exit(3);                  /* a child that fails with code 3 */
waitpid(pid, &status, 0);     printf("job 2 exit status %d\n",
WEXITSTATUS(status));     return 0;
}
```

Expected output:

```
shell: running job 1
42
job 1 exit status 0 shell: running job 2 job 2 exit status 3
```

Explain why stdout is flushed before each fork(). Then remove the second fflush(stdout), change _exit(3) to exit(3), redirect the output to a file and explain why "shell: running job 2" now appears twice.

Alternative Concept and Complexity Analysis

If commands were run with posix_spawn() or with threads instead of fork() followed by exec(), what would the solution be? Evaluate the time to start 1,000 commands, the memory copied or shared (copy-on-write), and the isolation between commands.

Evidence Expected

A working program that runs commands like a shell, with process diagrams and a reasoned evaluation.

TASK 8 – Evaluate CPU States and Protection — 3 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the hardware protects the hospital system.

Discuss:

- User mode and kernel mode, privileged instructions and the mode bit
- Interrupts, traps and the timer interrupt
- What would happen to the monitors if any program could disable interrupts
- How the hardware stops one program from reading another program's memory

Alternative Concept and Complexity Analysis

If the monitors used polling (checking the sensor in a loop) instead of interrupts, what would the solution be? Evaluate the worst-case delay before an abnormal reading is noticed (the polling interval versus the interrupt latency) and the CPU time and power wasted by polling.

Evidence Expected

A reasoned explanation of CPU modes, interrupts and protection, applied to the monitors.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how data moves between storage levels and devices.

Discuss:

- The levels of the memory hierarchy (registers, cache, main memory, SSD, hard disk, network storage) with typical access times and sizes
- Where MRI images, patient records and monitor readings should be kept
- Programmed I/O, interrupt-driven I/O and DMA, and which suits the MRI scanner • The effect of caching on the speed of record look-up

Alternative Concept and Complexity Analysis

If the MRI scanner transferred images with interrupt-driven I/O (one interrupt per word) instead of DMA, what would the solution be? Evaluate the number of interrupts and the CPU time for a 500 MB image, and the memory buffers needed in each method.

Evidence Expected

A justified data-placement plan and I/O method for each device, using typical access times.

TASK 10 – Measure and Compare 4 Marks

Bloom's Level: K5 – Evaluate

Measure the behaviour of your programs on a Linux system.

Discuss:

- The number of system calls made by your copy program for different buffer sizes (strace -c)
- The time spent in user mode and in kernel mode (the time command)
- The time to create 1,000 processes with fork() and to create 1,000 threads
- How the measurements support or contradict your earlier judgements

Alternative Concept and Complexity Analysis

If threads were created instead of processes for the measurements, what would the result be? Evaluate the creation time for 1,000 threads versus 1,000 processes and the memory used by each, and explain the difference using what each one shares.

Evidence Expected

Measured results in tables, compared with expectations, with explained differences.

TASK 11 – Recommend and Defend the Operating-System Plan 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the complete operating-system plan for the hospital.

Discuss:

- The type and structure of operating system for each environment
- The services, protection and I/O methods each will use
- The limitations and risks of your plan
- Whether the plan remains suitable for the group of three hospitals

Your recommendation must be based on **evidence from your own analysis and measurements**.

Alternative Concept and Complexity Analysis

If the whole system were moved to virtual machines on two large servers, instead of separate physical machines for each environment, what would the solution be? Evaluate the effect on response time for the monitors, the memory and processor overhead of virtualisation, and the effect on reliability.

Evidence Expected

A technically justified plan supported by analysis and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital adds **remote monitoring devices in patients' homes** that send readings over the internet, and moves patient records to a **cloud service** shared with the partner hospitals.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Identify which operating-system types, services and protection mechanisms are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your plan can support home devices and cloud records, considering response time, reliability and security.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe the revised plan (for example: an embedded real-time OS on home devices, a distributed system for shared records).

Evidence Expected

An appropriate design response.

D. Evidence-Based Defence – 2 Marks

Defend your decision using evidence and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating evidence and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Requirements of Each Environment	K5 – Evaluate	4
Task 2 – Types of Operating Systems	K5 – Evaluate	3
Task 3 – Operating-System Structures	K5 – Evaluate	4
Task 4 – Operating-System Services	K5 – Evaluate	4

Task 5 – System Calls	K5 – Evaluate	4
Task 6 – Cost of System Calls	K5 – Evaluate	4
Task 7 – Process Control System Calls	K5 – Evaluate	4
Task 8 – CPU States and Protection	K5 – Evaluate	3
Task	**Bloom's Level**	**Marks**
Task 9 – Memory Hierarchy and I/O	K5 – Evaluate	4
Task 10 – Measurement and Comparison	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including requirement tables, diagrams, strace and time outputs, measurements and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

TABLE OF CONTENTS

- Task 1 – Requirements of Each Computing Environment
- Task 2 – Types of Operating Systems
- Task 3 – Operating-System Structures
- Task 4 – Operating-System Services
- Task 5 – System Calls
- Task 6 – Cost of System Calls (Complete Code)
- Task 7 – Process Control System Calls (Complete Code)
- Task 8 – CPU States and Protection
- Task 9 – Memory Hierarchy and I/O Systems
- Task 10 – Measurement and Comparison
- Task 11 – Final Operating-System Plan
- Task 12 – Day-4 Design Challenge
- Conclusion
- References
- Appendix – Suggested GitHub README and Execution Commands

TASK 1 – EVALUATE THE REQUIREMENTS OF EACH COMPUTING ENVIRONMENT (4 MARKS)

1.1 Scenario and evaluation criteria

The hospital requires a dependable computing environment for bedside monitors, nurses' tablets, doctors' workstations, a central patient-record database, an MRI imaging server and links to two partner hospitals. The most safety-critical requirement is that a bedside monitor raises an alarm within 10 milliseconds after detecting an abnormal reading. The database must remain continuously available, while tablets must operate through a clinical shift without excessive battery drain.

1.2 Ranked requirements

Environment	Ranked requirements (highest first)	Justification
Bedside patient monitors	1. Deterministic response within 10 ms; 2. Reliability; 3. I/O accuracy; 4. Security; 5. Low memory/power use	A delayed or missed alarm can directly affect patient safety. Predictable scheduling is more important than high throughput or a rich user interface.
Nurses' handheld tablets	1. Battery life; 2. Security/privacy; 3. Reliable wireless communication; 4. Responsive interface; 5. Moderate throughput	Tablets are mobile and used throughout a shift. They need encrypted access and efficient power management, but do not need hard real-time scheduling.
Doctors' desktop workstations	1. Security; 2. Availability; 3. Interactive response; 4. Compatibility; 5. Multitasking throughput	Doctors access patient records, imaging and clinical applications. Isolation and usability are essential; occasional response variation is acceptable outside emergency alarms.
Central patient-record database	1. Availability/fault tolerance; 2. Data integrity; 3. Security/access control; 4. Transaction throughput; 5. Backup/recovery	The records service must not stop during updates. Replication, backups, transaction control and fail-over reduce service interruption and data loss.
MRI imaging server	1. High throughput; 2. Parallel processing; 3. Large memory/I/O bandwidth; 4. Reliability; 5. Security	MRI workloads process large image datasets and can exploit multiple processors, fast storage and DMA-capable I/O.
Partner-hospital network	1. Confidentiality/integrity; 2. Availability; 3. Interoperability; 4. Network throughput; 5. Manageable latency	Records must be exchanged securely and consistently. Network delay is relevant, but it does not replace the local 10 ms alarm path.

1.3 Conflicting requirements and patient safety

High throughput and many concurrent users can compete with short response times for CPU, memory and I/O. Power-saving modes can reduce tablet energy use but may increase wake-up latency. Strong encryption and access checks add processing work, but removing them would expose sensitive medical records. The design should therefore separate the real-time alarm path from general-purpose hospital applications.

- Critical to patient safety: the 10 ms bedside alarm deadline, reliable sensor acquisition, correct alarm delivery and safe behaviour during network or server failure.
- Important but less time-critical: report generation, historical analytics, bulk image transfer and routine desktop multitasking.

1.4 Lessons from operating-system history

Batch systems are suitable for non-interactive tasks such as overnight analytics and scheduled backups. Time-sharing systems support many simultaneous users on central services. Personal-computer operating systems provide interactive desktop environments for doctors. Mobile operating systems add power management, wireless connectivity and touch interfaces for nurses' tablets. Real-time operating systems are needed where a deadline must be met predictably.

1.5 Alternative: one general-purpose OS everywhere

Using one general-purpose OS on every device would simplify software administration, but it is not an acceptable uniform solution. A general-purpose scheduler may have variable delays caused by competing tasks, paging, drivers or background services; therefore, its worst-case alarm response cannot be assumed to remain below 10 ms without a validated real-time configuration. A small monitor may also waste memory and storage running desktop services. Administration may initially appear simpler, but updates and security policies would have to be tested against very different hardware and safety requirements.

Comparison	Specialised OS plan	One general-purpose OS everywhere
Alarm timing	A validated real-time design can provide bounded response under stated assumptions.	No hard deadline guarantee by default; worst-case delay may be unbounded.
Device memory	Small embedded footprint on monitors; richer OS only where needed.	Unnecessary services increase memory/storage requirements on small devices.
Administration	Multiple platforms require specialist	Fewer platform families, but one

	maintenance and testing.	update may create risk across all device types.
Safety judgement	Separates critical control from non-critical workloads.	Risks interference between clinical alarms and general-purpose activity.

Evaluation: retain a specialised real-time platform for monitors and use appropriate mobile, desktop and server operating systems elsewhere. This is safer than forcing all environments onto one general-purpose OS.

TASK 2 – JUDGE THE TYPE OF OPERATING SYSTEM FOR EACH ENVIRONMENT (3 MARKS)

2.1 Evaluation of OS types

A mainframe OS supports large-scale, centralised transaction processing and many users. Desktop OSs support interactive personal workstations. Multiprocessor OSs manage several CPUs/cores in one machine. Distributed OSs coordinate networked computers, while clustered systems provide coordinated service and fail-over. Hard real-time systems must meet strict deadlines; soft real-time systems aim to meet timing goals but can tolerate occasional misses. Handheld OSs are designed for mobile devices, touch interfaces and power management.

Hospital component	Recommended OS type	Reason and rejected alternative
Bedside monitors	Hard real-time embedded OS	The alarm deadline is safety-critical. A general desktop OS is too large and does not guarantee timing by default.
Nurses' tablets	Handheld/mobile OS	Supports wireless networking, touch UI, encryption and power management. A desktop OS wastes resources and battery.
Doctors' workstations	Desktop OS	Provides clinical applications, multitasking, display and peripheral support. A minimal RTOS would lack required application services.
Patient-record database	Clustered server OS, with distributed replication	Fail-over and redundancy improve availability. One standalone server has a single point of failure.
MRI imaging server	Multiprocessor server OS	Parallel processing and high memory/I/O bandwidth support image reconstruction.
Partner-hospital records	Distributed-system services over secure network links	Enables controlled sharing across sites. A single-site mainframe alone does not provide geographic resilience.

2.2 Symmetric versus asymmetric multiprocessing

In symmetric multiprocessing (SMP), processors share memory and the OS can schedule work on any available processor. This is generally suitable for an MRI server because image-processing jobs can be divided across cores and load-balanced. In asymmetric multiprocessing (AMP), processors may have fixed roles, such as one master processor coordinating other processors. AMP can be useful in specialised embedded designs, but it is less flexible for a changing mix of imaging workloads.

2.3 Database cluster versus one large multiprocessor

A cluster uses multiple independent machines connected through a network and coordinated by clustering or database software. If one node fails, another node can take over, subject to replication, quorum and recovery design. A single large multiprocessor server can offer strong throughput, but failure of its motherboard, power supply, storage subsystem or OS may stop the whole service unless redundant components and external recovery are provided.

Factor	Cluster of smaller servers	One large multiprocessor server
Availability	Can fail over to a healthy node; fail-over is not necessarily instantaneous and must be tested.	Single system failure can interrupt service unless additional redundancy exists.
Cost	Can scale incrementally; network and replication add cost.	High initial cost and potentially expensive specialised hardware.
Throughput (time)	Can scale by adding nodes, subject to coordination and network overhead.	Can scale with processors/cores until memory bandwidth and contention limit gains.
Memory (space)	Memory is distributed across nodes; data replication consumes additional storage/memory.	Large shared-memory capacity may be available, but has a hardware ceiling.
Complexity	Requires cluster management, consistency, fail-over and monitoring.	Simpler topology, but larger single-system maintenance and failure risk.

Complexity note: for a workload of size n, ideal parallel processing on p processors may approach O(n/p) computation time, but real performance includes synchronisation and I/O overhead. Space is O(n) for the dataset; in a replicated cluster, total storage may increase with the replication factor r, approximately O(rn).

Decision: use a clustered, replicated database service for patient records and an SMP server for MRI processing. The two workloads have different priorities: availability and continuity for records, and parallel throughput for imaging.

TASK 3 – EVALUATE OPERATING-SYSTEM STRUCTURES (4 MARKS)

3.1 Structural alternatives

Structure	Description	Reliability / performance / updates
Simple / monolithic	Most OS services execute in one large kernel address space.	Fast direct service paths can reduce overhead; a faulty driver or kernel component can crash the whole system; changes may affect many components.
Layered	OS is organised into layers; each layer uses services of lower layers.	Clear interfaces support testing and maintenance; crossing layers can add overhead and strict layering may limit flexibility.
Microkernel	Kernel retains minimal mechanisms; services such as drivers or file systems run as separate user-space servers.	Fault isolation and modular updates are strong; IPC and context switches can add service latency.
Modular	A core kernel can load or unload modules such as drivers and file systems.	Flexible and efficient; a faulty kernel module can still compromise the kernel.
Hybrid	Combines ideas from monolithic, microkernel and modular designs.	Balances performance and modularity, but complexity can increase.

3.2 Application to the hospital

For bedside monitors, a microkernel-based or small modular real-time OS is preferred because a driver or service failure should be isolated where possible, and the kernel should remain small and predictable. The design must still be validated for its actual deadline and hardware. For the database server, a mature modular or hybrid server OS is suitable because it needs high I/O performance, file systems, networking, security controls and a broad ecosystem of storage and database drivers.

3.3 Examples of operating-system structures

- Linux: generally described as a monolithic but modular kernel; loadable kernel modules provide flexibility.
- Windows: commonly described as a hybrid kernel architecture with kernel-mode and user-mode components.
- Android: built on the Linux kernel with a layered software stack and application framework.
- FreeRTOS: a small real-time kernel intended for embedded systems; QNX is a microkernel-based real-time OS.

3.4 Alternative: monolithic monitor kernel versus microkernel

A monolithic monitor kernel can provide a service through a system call and direct in-kernel function path. A microkernel design may require a system call plus IPC/message exchange with a driver or service process. Therefore, a microkernel service path can have more context-switch and message-passing overhead. However, the microkernel can keep kernel memory smaller and isolate some service failures. In a monolithic kernel, a faulty driver running in kernel mode may crash or corrupt the whole system.

Evaluation factor	Monolithic kernel	Microkernel
Service-request time	Often fewer transitions and lower overhead.	May involve IPC and extra context switches; latency must be measured.
Kernel memory	More services and drivers may reside in kernel space.	Small core kernel; services run outside the kernel.
Fault impact	Kernel-mode driver failure can affect the whole OS.	User-space service failure can be isolated/restarted, depending on design.
Monitor decision	Not preferred unless the implementation is proven safe and deterministic.	Preferred candidate for separation and fault containment, subject to real-time validation.

Complexity comparison: for k service requests, a simple monolithic path has O(k) request operations with a lower constant overhead. A microkernel path also has O(k) asymptotic request count, but a higher constant factor from IPC and context switches. Kernel memory is implementation-dependent; it is not valid to claim a universal byte count without measuring the selected OS build.

Decision: choose a small real-time microkernel or similarly isolated embedded architecture for monitors; choose a mature modular/hybrid server OS for the database.

TASK 4 – EVALUATE THE SERVICES AN OPERATING SYSTEM MUST PROVIDE (4 MARKS)

4.1 Core OS services

OS service	Purpose	Hospital application
User interface	Accepts user commands and displays results.	Rich interface on tablets/desktops; minimal status/alarm interface on monitors.
Program execution	Loads, starts, schedules and terminates programs.	Runs monitoring firmware, clinical apps, database engine and image reconstruction.
I/O operations	Controls device input/output.	Sensor sampling, displays, scanners, storage and network interfaces.
File-system management	Creates, reads, writes and protects files.	Needed on servers and workstations; monitors may use a small configuration/log store.
Communication	Supports communication between processes and machines.	Secure hospital network and partner-hospital exchange.
Error detection	Detects hardware/software and I/O errors.	Essential for sensor faults, storage errors and network failures.
Resource allocation	Allocates CPU, memory, devices and storage.	Prioritises alarm processing and controls server workloads.
Accounting/monitoring	Records resource use and activity.	Audit logs, capacity planning and security monitoring.
Protection/security	Controls access and isolates processes/data.	Authentication, authorisation, encryption and patient-record isolation.

4.2 Services for bedside monitors

A monitor should retain deterministic program execution, sensor/display I/O, error detection, essential communication, resource allocation, watchdog/timer support and strong device authentication. A rich desktop user interface, general-purpose office applications, broad multimedia services and unnecessary file-system features can be excluded. Logging should be bounded and should not delay alarm generation.

4.3 Essential database services and patient-record isolation

The database server requires reliable process execution, high-performance storage I/O, journaling or equivalent crash recovery, file-system support, networking, memory management, resource allocation, accounting, audit logging, access control and security updates. Patient-record isolation

should be enforced through authenticated identities, role-based permissions, database-level authorisation, process isolation, encryption in transit and at rest, and audit trails. One user's query must not be able to access another patient's records unless the user has explicit permission.

4.4 Alternative: full desktop OS on tablets

A full desktop OS may include background services, indexing, print services, desktop effects and general-purpose applications that a clinical tablet does not need. These services increase storage and memory use and may lengthen startup or wake-up time. They also consume energy through background CPU, radio and storage activity. A reduced handheld OS can disable unnecessary services and use mobile power-management policies.

Metric	Reduced handheld OS	Full desktop OS on tablet
Space	Smaller service set and lower baseline memory/storage.	More services and applications increase storage and memory demand.
Time	Faster boot/wake is possible with fewer background tasks.	More startup tasks can increase boot and update time.
Battery	Mobile power management and reduced background work improve endurance.	Background services may increase power consumption.
Security	Smaller attack surface if maintained correctly.	Larger software surface requires more patching and policy management.

Complexity note: startup work is approximately O(s) in the number of services/tasks initialised; actual time depends on I/O and dependency ordering. Space is O(m) in the total memory footprint of active services. These are qualitative models; actual boot time, RAM and battery life must be measured on the chosen tablet.

TASK 5 – EXAMINE SYSTEM CALLS (4 MARKS)

5.1 Categories of system calls

Category	Examples	Hospital use
Process control	fork(), execve(), waitpid(), exit()	Start database utilities, manage clinical application processes.
File management	open(), read(), write(), close(), stat()	Read patient records, store logs and MRI images.
Device management	ioctl(), read(), write()	Configure sensors, displays, storage and device drivers.
Information maintenance	getpid(), clock_gettime(), uname()	Obtain process, time and OS information for monitoring/auditing.
Communication	socket(), connect(), send(), recv(), pipe()	Communicate with partner hospitals and local services.
Protection	chmod(), chown(), access() and security-related interfaces	Manage permissions and protect patient information.

5.2 What happens during read()

1. An application calls the C library wrapper read(fd, buffer, count).
2. The wrapper places the system-call number and arguments in the architecture-defined registers or memory locations.
3. A syscall/trap instruction transfers execution from user mode to kernel mode.
4. The kernel validates the file descriptor, checks permissions and verifies the user buffer range.
5. The file-system and device layers obtain the requested data, possibly waiting for storage or a device.
6. The kernel copies or maps the result to the user buffer, updates file position and prepares a return value.
7. The CPU returns to user mode; the wrapper returns the number of bytes read or an error indication.

5.3 Parameter-passing methods

System-call arguments may be passed directly in CPU registers, placed in a block in user memory with a pointer passed in a register, or placed on the user stack according to the processor ABI and OS interface. The kernel must validate pointers and sizes before accessing user memory.

5.4 Using strace

On Linux, strace records system calls made by a process. For example, `strace -e trace=openat,read,write,close ./program` shows file opening, data transfer and closing. `strace -c ./program` summarises call counts and time. The output should be captured from the student's own Linux environment.

5.5 Alternative: C library buffered I/O versus direct system calls

The C library functions fopen(), fread() and fwrite() provide a stream abstraction and normally buffer data in user space. With a typical 4096-byte library buffer, a file of n bytes may require approximately O(n/4096) underlying read/write calls, rather than O(n/b) calls when the application uses a direct buffer of b bytes. For a 10,000-byte file, the rough count is about three data reads plus an end-of-file read, depending on buffering and implementation. The library also consumes buffer space, commonly O(4096) per stream for a 4 KiB buffer.

Evaluation: buffered stdio is convenient and reduces small system calls. Direct read()/write() is useful when exact buffer sizes and I/O behaviour must be controlled. The actual system-call count should be confirmed with strace, not assumed from the C function names.

TASK 6 – EVALUATE THE COST OF SYSTEM CALLS (4 MARKS)

6.1 Why system calls cost more than ordinary function calls

An ordinary function call generally changes control flow within the same process and privilege level. A system call crosses the user/kernel protection boundary, performs validation and permission checks, may acquire kernel locks, may invoke file-system or device code, and may block or cause scheduling. This work makes a system call more expensive than a simple user-space function call.

6.2 Effect of buffer size

For a file of n bytes and a read buffer of b bytes, the approximate number of successful data read calls is ceil(n/b), plus one final read that returns end-of-file. For n = 10,000 bytes, the expected counts in the assignment are: b = 1 gives 10,001 calls; b = 512 gives 21 calls; b = 4096 gives 4 calls. Larger buffers reduce system-call overhead but use more memory. A backup program should use a buffer large enough to amortise call overhead without consuming excessive memory; the best value should be selected from measured results.

6.3 Complete C program

Save as Task06/copy_buffer.c. The program completes the supplied TODO using read() and write(), counts every read attempt, handles partial writes and validates the buffer size.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>

long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in, out;
    long calls = 0;
    ssize_t n;

    if (bufsize == 0 || bufsize > sizeof(buf)) {
        errno = EINVAL;
        return -1;
    }

    in = open(src, O_RDONLY);
    if (in < 0) return -1;

    out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) {
        close(in);
        return -1;
    }
```

```
    for (;;) {
        calls++;
        n = read(in, buf, bufsize);
        if (n == 0) break;
        if (n < 0) {
            if (errno == EINTR) { calls--; continue; }
            close(in); close(out);
            return -1;
        }

        ssize_t written = 0;
        while (written < n) {
            ssize_t w = write(out, buf + written, (size_t)(n - written));
            if (w < 0) {
                if (errno == EINTR) continue;
                close(in); close(out);
                return -1;
            }
            written += w;
        }
    }

    if (close(in) < 0) { close(out); return -1; }
    if (close(out) < 0) return -1;
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");
    if (f == NULL) { perror("fopen"); return 1; }

    for (int i = 0; i < 10000; i++) {
        if (fputc('A' + i % 26, f) == EOF) {
            perror("fputc"); fclose(f); return 1;
        }
    }
    if (fclose(f) != 0) { perror("fclose"); return 1; }

    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++) {
        long result = copy_file("records.dat", "copy.dat", sizes[i]);
        if (result < 0) {
            perror("copy_file");
            unlink("records.dat"); unlink("copy.dat");
            return 1;
        }
        printf("buffer %4zu bytes: %5ld read() calls\n",
               sizes[i], result);
    }

    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

6.4 Compile and run

```
gcc -std=c11 -Wall -Wextra -O2 copy_buffer.c -o copy_buffer
./copy_buffer
strace -c ./copy_buffer
strace -e trace=read,write,openat,close ./copy_buffer
```

Expected program output from the assignment specification:

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Note: the program counts read() calls made inside copy_file(). strace -c also counts other reads performed by the C runtime or dynamic loader, so compare the relevant read() entries and understand that the total process count may be higher.

6.5 Alternative: memory-mapped files (mmap)

With mmap(), a process maps a file into its virtual address space and accesses it as memory. The application can avoid repeated read() calls and explicit copying through a user buffer. Mapping and unmapping require a small number of system calls, but page faults still occur as pages are first accessed, and dirty pages must be written back. For an n-byte file, a buffered read/write design has O(n) data processing time, O(n/b) read/write calls and O(b) user buffer space. An mmap design has O(n) data access in the full scan, a small constant number of mapping calls plus page faults, and O(n) virtual address space; resident physical memory depends on the working set and OS paging.

Evaluation: mmap can reduce explicit copying and syscall frequency for suitable workloads, but mapping a very large file reserves a large virtual range and page faults can produce latency variation. For predictable backup throughput, a measured 64 KiB–1 MiB read/write buffer is a practical candidate to test; do not claim it is optimal until measured on the target storage.

6.6 Evidence to insert

Buffer size	Program read() calls	strace read() count	Elapsed time	User time	System time
1 byte	10001	[Insert measured value]	[Insert]	[Insert]	[Insert]
512 bytes	21	[Insert measured value]	[Insert]	[Insert]	[Insert]
4096 bytes	4	[Insert measured value]	[Insert]	[Insert]	[Insert]
Optional larger buffer	[Measure]	[Insert]	[Insert]	[Insert]	[Insert]

Attach a terminal screenshot showing the command and output. Do not fill measurement cells with estimated or invented values.

TASK 7 – EVALUATE PROCESS CONTROL SYSTEM CALLS (4 MARKS)

7.1 Process-control calls

- fork() creates a child process by duplicating the calling process context; memory is commonly implemented using copy-on-write.
- exec() replaces the current process image with a new program. The process ID remains the same.
- wait() / waitpid() lets a parent collect a child's termination status and release its process-table entry.
- exit() terminates a process and makes its status available to the parent; _exit() terminates without running C library exit handlers or flushing stdio buffers.

A shell typically parses a command, forks a child, executes the requested program in the child and waits for it when running a foreground command. The shell obtains the exit status using waitpid() and macros such as WIFEXITED() and WEXITSTATUS(). A zombie is a terminated child whose status has not yet been collected by its parent. An orphan is a child whose parent terminates first; it is re-parented to a system process that eventually collects its status.

7.2 Complete C program

Save as Task07/shell_jobs.c. This fills the supplied TODO and includes error checks while preserving the requested command sequence.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>

static int wait_for_child(pid_t pid, int *status)
{
    while (waitpid(pid, status, 0) < 0) {
        if (errno == EINTR) continue;
        return -1;
    }
    return 0;
}

int main(void)
{
    pid_t pid;
    int status;

    printf("shell: running job 1\n");
    fflush(stdout);

    pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
```

```
    if (pid == 0) {
        execlp("expr", "expr", "6", "*", "7", (char *)NULL);
        perror("execlp");
        _exit(127);
    }

    if (wait_for_child(pid, &status) < 0) {
        perror("waitpid");
        return 1;
    }
    if (WIFEXITED(status))
        printf("job 1 exit status %d\n", WEXITSTATUS(status));
    else
        printf("job 1 terminated abnormally\n");

    printf("shell: running job 2\n");
    fflush(stdout);

    pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) _exit(3);

    if (wait_for_child(pid, &status) < 0) {
        perror("waitpid");
        return 1;
    }
    if (WIFEXITED(status))
        printf("job 2 exit status %d\n", WEXITSTATUS(status));
    else
        printf("job 2 terminated abnormally\n");

    return 0;
}
```

7.3 Compile and run

```
gcc -std=c11 -Wall -Wextra shell_jobs.c -o shell_jobs
./shell_jobs
```

Expected output when expr is available in PATH:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

7.4 Why stdout is flushed before fork()

When stdout is connected to a terminal, it is usually line-buffered, but when redirected to a file it is usually fully buffered. If text remains in the stdio buffer when fork() occurs, the buffer is copied into both parent and child. If the child calls exit(), it flushes its copy; the parent later flushes its own copy, so the same line can appear twice. fflush(stdout) empties the buffer before fork(), preventing this duplication. _exit() does not flush inherited stdio buffers, which is why the supplied child uses _exit(3).

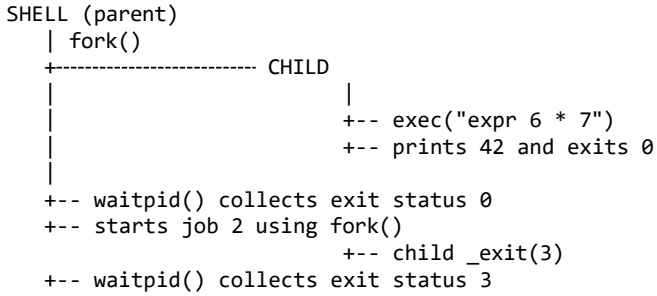
7.5 Alternative: posix_spawn() or threads

posix_spawn() combines process creation and program execution through a library interface and may be more efficient on some systems. Threads share the same address space and process resources, so they can be cheaper to create, but they do not provide the same memory isolation as separate processes. A command that must execute an independent program and be isolated should normally use a process, not a thread.

Approach	Time for 1,000 launches	Space / isolation
fork() + exec()	O(1000) launch operations; actual elapsed time depends on process setup and executable loading.	Copy-on-write initially shares pages; separate address spaces provide strong isolation.
posix_spawn()	O(1000) launches; may reduce setup overhead on some implementations.	Separate process address spaces; strong isolation.
1,000 threads	O(1000) thread creations; often lower setup overhead than processes.	Threads share process memory; each needs a stack and thread metadata; weaker fault isolation.

Measurement requirement: time the creation of 1,000 child processes and 1,000 threads on the same Linux machine. Report compiler, OS, CPU and exact method. Big-O is O(k) creation operations for k children/threads; per-task memory is implementation-dependent. For processes, copy-on-write shares unchanged pages but page tables and process metadata are still required. For threads, stacks and thread-control blocks add memory, while the address space is shared.

7.6 Process diagram



TASK 8 – EVALUATE CPU STATES AND PROTECTION (3 MARKS)

8.1 User mode and kernel mode

The CPU operates in at least two privilege levels. User mode restricts application programs from executing privileged instructions or directly accessing protected kernel memory. Kernel mode allows the OS to manage hardware, memory mappings, interrupts and devices. A hardware mode bit or privilege state indicates the current execution level. A system call or interrupt transfers control to trusted kernel code; a controlled return restores user mode.

8.2 Interrupts, traps and timer interrupts

- An interrupt is an asynchronous event, commonly generated by hardware such as a network interface or timer.
- A trap/exception is a synchronous event caused by the current instruction, such as a system call or invalid operation.
- A timer interrupt allows the OS scheduler to regain control and prevent one process from monopolising the CPU.

If ordinary programs could disable interrupts, a faulty or malicious program could prevent the OS from servicing timers and device events. A bedside monitor might fail to process sensor events or raise alarms on time. Therefore, disabling interrupts must be restricted to privileged kernel code and for carefully bounded intervals.

8.3 Memory protection

The memory-management unit (MMU) uses page tables and access permissions to restrict which virtual memory pages a process can read, write or execute. A process cannot normally read another process's private pages or kernel memory. Invalid access causes a fault that the OS handles, preserving isolation between clinical applications and patient data.

8.4 Alternative: polling versus interrupts

With polling, the monitor repeatedly checks a sensor at a fixed interval T. If an abnormal event occurs just after a poll, the detection delay can approach T, so the worst-case detection delay is approximately the polling interval plus processing time. With interrupts, the sensor or controller signals an event; response delay is approximately interrupt latency plus handler and scheduling time.

Interrupts can reduce wasted CPU cycles and power when events are infrequent. However, interrupt storms and long handlers can also affect timing, so the implementation must be bounded and tested.

For a 10 ms alarm requirement, choose interrupt-driven acquisition or a validated periodic sampling design whose worst-case detection and processing time is demonstrably below 10 ms. Do not assume interrupts automatically guarantee the deadline.

TASK 9 – EVALUATE THE MEMORY HIERARCHY AND I/O SYSTEMS (4 MARKS)

9.1 Memory hierarchy

The hierarchy places small, fast storage close to the CPU and larger, slower storage farther away. Values below are broad illustrative ranges; actual values depend on processor, memory technology, storage device, network and workload.

Level	Typical access time (approx.)	Typical capacity / size	Hospital use
CPU registers	Sub-nanosecond to a few ns	Bytes to a few KB per core	Current instructions, addresses and arithmetic operands.
CPU cache (L1/L2/L3)	About 1–30 ns	Tens of KB to tens of MB	Frequently used code, database indexes and hot data.
Main memory (DRAM)	About 50–100 ns	GBs to TBs in servers	Active database buffers, application working sets and image-processing data.
NVMe SSD	About 10–100 µs for common I/O paths	Hundreds of GB to many TB	Fast patient-record storage, active MRI datasets and logs.
Hard disk drive	About 3–15 ms for random access	TBs to tens of TB	Large archives and lower-cost bulk storage.
Network storage	Sub-ms to tens of ms or more	Scalable shared capacity	Replicated records, backups and partner-hospital shared data.

9.2 Data placement

Data	Recommended placement	Reason
MRI images	Active scans in RAM/cache and fast SSD; long-term images in redundant archive/storage.	Large files need high bandwidth during processing and durable archival afterwards.
Patient records	Database memory cache plus redundant SSD-backed storage and replicated backup.	Fast look-up and transaction processing with recovery and high availability.
Bedside monitor readings	Local bounded buffer for immediate alarm processing; transmit summaries/events to secure central storage.	Alarm processing must continue even if the network is temporarily unavailable.

9.3 Programmed I/O, interrupt-driven I/O and DMA

- Programmed I/O: the CPU repeatedly checks device status and moves data. It is simple but wastes CPU time for large transfers.
- Interrupt-driven I/O: a device interrupts the CPU when it needs service or a transfer completes. It avoids constant polling but may generate many interrupts.
- Direct Memory Access (DMA): a DMA controller transfers blocks between a device and memory with limited CPU involvement. The CPU configures the transfer and handles completion/error events.

DMA is generally the best fit for large MRI image transfers because it can move blocks efficiently without interrupting the CPU for every word. The scanner and host must support a compatible DMA interface and safe buffer management.

9.4 Alternative: interrupt per word versus DMA for a 500 MB image

Assume a 500 MB image and one interrupt for each 4-byte word. The number of interrupts would be 500,000,000 / 4 = 125,000,000 interrupts using decimal MB. With 500 MiB, the count would be 131,072,000. Handling that many interrupts would impose substantial CPU overhead and could delay other tasks. DMA instead transfers blocks and typically needs a small number of setup/completion interrupts.

Method	Interrupt / CPU work	Buffer and memory implication
Interrupt per 4-byte word	About 125 million interrupts for 500 MB (decimal); very high CPU overhead.	Small transfer unit, but enormous event-handling overhead.
DMA block transfer	A few setup/completion or block-level interrupts, depending on controller and driver.	Requires one or more suitably sized DMA buffers; buffers can be reused in a ring or pool.

Complexity comparison: both methods transfer O(n) bytes. Interrupt-per-word handling creates O(n/w) interrupts, where w is bytes per word. DMA reduces interrupt count to O(n/B) for block size B, often with a much larger B. DMA buffer space is O(B × q) for q queued buffers; exact values depend on throughput and device constraints.

9.5 Caching

A database cache keeps frequently accessed records and indexes in RAM, reducing repeated SSD or network access. Cache design must account for consistency, invalidation and patient-data security. A

cache hit improves response time, but stale data must not be returned where clinical correctness requires the latest committed record.

TASK 10 – MEASURE AND COMPARE (4 MARKS)

10.1 Measurement procedure

Run all measurements on the same Linux machine under similar conditions. Record the distribution and version, kernel version, CPU model, RAM, GCC version and whether the machine was otherwise busy. Repeat each test several times and report median or mean with the method clearly stated.

10.2 System-call count and time

```
strace -c ./copy_buffer
/usr/bin/time -v ./copy_buffer
strace -c -e trace=read,write ./copy_buffer
```

Use the program output to confirm the expected read-call counts. Use strace to observe the calls made by the complete process. Use time to record elapsed time and user/system CPU time. Explain that small test files may be dominated by startup and measurement overhead, so repeat with a larger file if permitted.

10.3 Process versus thread creation experiment

Create a small benchmark that launches and waits for 1,000 child processes using fork()/waitpid(), then create and join 1,000 POSIX threads using pthread_create()/pthread_join(). Keep the work done by each child/thread minimal and identical. Compile the thread test with -pthread. Record failures rather than silently counting them as successful launches.

10.4 Results table – fill with your own measured values

Experiment	Run 1	Run 2	Run 3	Reported result (median)
read() calls, 1-byte buffer	[ ]	[ ]	[ ]	[ ]
read() calls, 512-byte buffer	[ ]	[ ]	[ ]	[ ]
read() calls, 4096-byte buffer	[ ]	[ ]	[ ]	[ ]
copy program elapsed time	[ ]	[ ]	[ ]	[ ]
copy program user time	[ ]	[ ]	[ ]	[ ]
copy program system time	[ ]	[ ]	[ ]	[ ]
1,000 process creation + wait	[ ]	[ ]	[ ]	[ ]

1,000 thread creation + join	[ ]	[ ]	[ ]	[ ]

10.5 Interpretation

The expected trend is that larger buffers reduce read() calls. The elapsed-time improvement may be smaller than the syscall-count reduction because storage caching, runtime startup, scheduling and measurement noise also affect the result. Thread creation is often faster and uses less per-task memory than process creation because threads share an address space; however, actual results depend on the OS, libraries, stack sizes and system load. State whether your measurements support or contradict these expectations and explain any unexpected result.

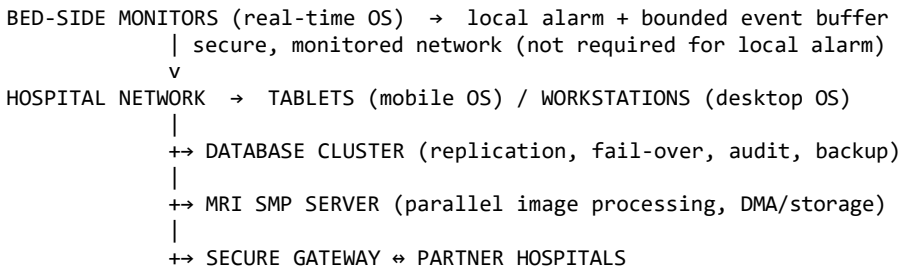
Do not substitute values from another computer. Attach screenshots of the exact commands and outputs and preserve raw logs in Task10 in the GitHub repository.

TASK 11 – RECOMMEND AND DEFEND THE OPERATING-SYSTEM PLAN (4 MARKS)

11.1 Complete recommended plan

Environment	OS type / structure	Services, protection and I/O
Bedside monitors	Hard real-time embedded OS; small microkernel or validated modular RTOS.	Bounded scheduling, sensor interrupts, watchdog, minimal file/log services, authenticated communication and memory protection. Alarm processing remains local.
Nurses' tablets	Handheld/mobile OS.	Touch UI, Wi-Fi, secure authentication, encryption, device management, power management and cached read-only information where appropriate.
Doctors' workstations	Desktop OS with hybrid/modular architecture.	Clinical applications, secure user accounts, endpoint protection, encrypted network access, display and peripheral support.
Patient-record database	Clustered server OS with replicated database service.	Transactions, journaling, access control, audit logs, redundant SSD storage, backups, health checks and tested fail-over.
MRI imaging server	SMP multiprocessor server OS with modular kernel.	Parallel processing, large RAM, high-speed storage, DMA-capable I/O, job scheduling and secure image storage.
Partner-hospital link	Distributed services over secure network.	Mutual authentication, encryption, access policy, audit, replication and controlled interoperability.

11.2 System-level architecture



11.3 Risks and limitations

- Multiple OS families increase patching, testing, training and support effort.

- A cluster does not guarantee zero downtime; replication lag, split-brain, quorum loss and fail-over bugs must be addressed.
- Real-time performance depends on hardware, driver quality, configuration, workload bounds and verification. The 10 ms deadline must be tested end-to-end.
- Network outages must not disable local alarm generation. Offline buffering and recovery procedures are required.
- Patient data requires least-privilege access, encryption, audit logging, backup and tested restoration.

11.4 Scaling to three hospitals

The design can scale to three hospitals by using secure inter-site networking, replicated or federated record services, standardised identity and access management, consistent audit policies and tested disaster recovery. Local bedside alarms remain independent of the wide-area link. Capacity planning should include peak concurrent users, image-transfer volume, replication bandwidth and recovery objectives.

11.5 Alternative: all virtual machines on two large servers

Virtualisation can consolidate desktop, database and non-real-time application workloads and simplify deployment. However, running bedside monitor control as virtual machines on two shared servers would introduce hypervisor scheduling, virtual I/O and shared-resource contention into a safety-critical path. The 10 ms deadline should not depend on general-purpose VM scheduling unless a certified/validated real-time virtualisation platform and end-to-end timing evidence are available. Two servers also create concentration risk: failure or maintenance of a host may affect many services. Memory overhead includes guest OS memory plus hypervisor and virtual-device overhead; processor overhead comes from scheduling and virtual I/O.

Decision: use virtualisation selectively for non-real-time services, but keep bedside alarm processing on dedicated embedded real-time devices. Keep database availability through a properly designed cluster rather than relying only on two consolidated hosts.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN (8 MARKS)

A. Impact analysis (2 marks)

The addition of home remote-monitoring devices changes the system boundary. Devices now operate over consumer networks and may have intermittent connectivity, limited battery capacity and less controlled physical security. The cloud-based records service adds geographically distributed storage, shared access across partner hospitals, identity federation, network security and data-governance requirements.

Changed area	Impact on OS type/services/protection
Home monitoring device	Needs a small embedded OS with efficient power management, secure boot/update, sensor I/O, local buffering and a safe local alarm strategy where clinically required.
Internet connectivity	Requires reliable network services, encrypted communication, authentication, retry/backoff and offline queueing.
Cloud records	Requires server/cloud OS services, virtualisation or container support, distributed storage, replication, monitoring, backup and recovery.
Partner-hospital sharing	Requires identity federation, role-based access, audit trails, data integrity and controlled API access.
Privacy and security	Requires encryption in transit/at rest, device identity, key management, patching and protection against lost or compromised devices.

B. Evaluation of the original design (2 marks)

The original design remains appropriate for in-hospital bedside monitors, tablets, workstations, the MRI server and a clustered record service. However, it is incomplete for home monitoring and cloud records. Home devices cannot rely on a stable hospital LAN, and a remote network round trip cannot be assumed to meet the local 10 ms alarm deadline. Cloud services improve geographic accessibility and elasticity but introduce WAN dependency, shared-responsibility security, availability and data-location considerations. Therefore, the original plan should be modified rather than retained unchanged.

C. Adaptation or redesign (2 marks)

Layer	Revised design

Home sensor/device	Embedded real-time OS or small embedded OS; local sampling, threshold detection, local alarm where specified, bounded data buffer, secure boot and signed updates.
Home gateway/mobile app	Mobile or gateway OS; authenticated pairing, encrypted upload, store-and-forward when offline, battery/network monitoring.
Cloud records platform	Distributed cloud service with redundant instances, replicated database/storage, backups, monitoring and tested disaster recovery.
Hospital edge/network	Secure gateway, device identity and certificate management, network segmentation and local buffering.
Partner hospitals	Federated identity, authorised APIs, audit trails and data exchange standards agreed by the hospitals.

D. Evidence-based defence (2 marks)

The revised design preserves the safety-critical principle: local alarm generation must not depend on a cloud response. A home device can detect a configured abnormal reading locally and transmit the event to the cloud when connectivity is available. Store-and-forward buffers protect against temporary network loss. The cloud service supports shared records and scaling, while encryption, authentication, least privilege and audit reduce privacy risks. The trade-off is greater operational complexity: cloud services require continuous monitoring, key management, vendor/service-level review, recovery testing and careful handling of delayed or duplicate events.

Final decision: MODIFY the original design. Retain dedicated real-time embedded systems for local monitoring and the secure clustered record architecture, while adding secure home-device connectivity and a distributed cloud records layer. This design separates hard real-time safety functions from best-effort wide-area communication and is more suitable for a three-hospital group with remote patients.

12.1 Design principles to validate

- Local alarm detection and alarm output continue during internet/cloud outage.
- Remote readings are timestamped, authenticated and protected in transit.
- Offline events are buffered and transmitted safely after reconnection.
- Cloud records use replication, backup, access control and audit logging.
- Duplicate, delayed and out-of-order readings are handled explicitly.
- Device firmware and cloud services receive authenticated, tested updates.

- End-to-end latency, reliability and recovery objectives are measured and documented.

CONCLUSION

The smart hospital should not use one general-purpose operating system for every device. Each environment has different priorities: bedside monitors require predictable real-time response; tablets require mobility and battery efficiency; workstations require interactive clinical applications; the patient-record service requires availability, integrity and security; and the MRI server requires multiprocessor throughput and high-speed I/O. A clustered database and secure distributed services support growth to three hospitals.

Operating-system structures and services must be selected according to the risk and workload of each environment. System calls provide controlled access to kernel services, while process control, CPU privilege levels, memory protection, caching and DMA contribute to safe and efficient operation. The completed C programs demonstrate how buffer size affects read-call counts and how fork(), exec() and waitpid() support shell-like process management.

The design challenge requires modification rather than complete replacement. Home devices should retain local safety functions and use secure store-and-forward communication, while cloud records should use distributed, replicated services. Before submission, all code must be executed on the student's Linux system and the genuine measurements, screenshots and GitHub link must be added.

REFERENCES

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Boston, MA, USA: Pearson, 2015.

[3] W. Stallings, Operating Systems: Internals and Design Principles, 9th ed. Hoboken, NJ, USA: Pearson, 2018.

[4] IEEE, IEEE Standard for Information Technology—Portable Operating System Interface (POSIX), IEEE Std 1003.1-2017, 2018.

[5] Linux man-pages project, "read(2) — Linux manual page," man7.org. Accessed: Nov. 3, 2026.

[6] Linux man-pages project, "write(2) — Linux manual page," man7.org. Accessed: Nov. 3, 2026.

[7] Linux man-pages project, "fork(2) — Linux manual page," man7.org. Accessed: Nov. 3, 2026.

[8] Linux man-pages project, "execve(2) — Linux manual page," man7.org. Accessed: Nov. 3, 2026.

[9] Linux man-pages project, “wait(2) — Linux manual page,” man7.org. Accessed: Nov. 3, 2026.

[10] Linux man-pages project, “mmap(2) — Linux manual page,” man7.org. Accessed: Nov. 3, 2026.

[11] Linux man-pages project, “strace(1) — Linux system call tracer,” man7.org. Accessed: Nov. 3, 2026.

[12] FreeRTOS, “FreeRTOS Kernel Book and Documentation,” freertos.org. Accessed: Nov. 3, 2026.

[13] QNX Software Systems, “QNX Neutrino RTOS System Architecture,” QNX documentation. Accessed: Nov. 3, 2026.

APPENDIX – SUGGESTED GITHUB README AND EXECUTION RECORD

A. Repository structure

```
CSA04-<RegisterNumber>-Assignment1/
  README.md
  Task01/
  Task02/
  Task03/
  Task04/
  Task05/
  Task06/copy_buffer.c
  Task06/terminal_output.txt
  Task07/shell_jobs.c
  Task07/terminal_output.txt
  Task08/
  Task09/
  Task10/benchmark source and raw logs
  Task11/
  Task12/
  screenshots/
```

B. Useful environment commands

```
gcc --version
cat /etc/os-release
uname -a
gcc -std=c11 -Wall -Wextra -O2 copy_buffer.c -o copy_buffer
./copy_buffer
strace -c ./copy_buffer
/usr/bin/time -v ./copy_buffer
gcc -std=c11 -Wall -Wextra shell_jobs.c -o shell_jobs
./shell_jobs
```